



PENSAMENTO COMPUTACIONAL • AULA 14 DE 15

Complexidade de Algoritmos

Da função de custo ao Big O, com experimentos em Julia + Pluto — roteiro visual para aula ou minicurso.

Objetivos de aprendizagem

mapa da aula

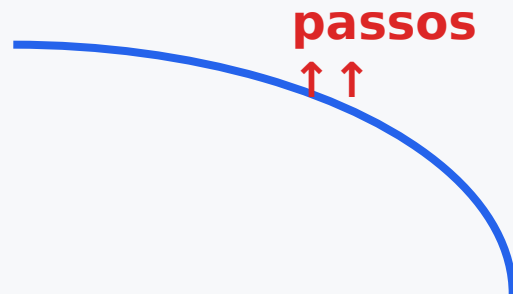
- 1 Construir a função de custo $T(n)$ a partir da contagem de passos.
- 2 Interpretar sequência, decisão, repetição e recursão como contribuições distintas ao custo.
- 3 Diferenciar eficiência prática, escalabilidade e classificação assintótica.
- 4 Usar Julia + Pluto para testar hipóteses com gráficos e sliders.

A pergunta central

escala

Um algoritmo que funciona para 100 entradas ainda será viável para 100 mil?

$n \uparrow$



A análise de complexidade não pergunta apenas “funciona?”. Ela pergunta “como o trabalho cresce?”.

Essa pergunta antecede o tempo em segundos, porque segundos mudam com a máquina; passos pertencem ao algoritmo.

Passos, não segundos

modelo de custo

tempo medido

Depende de processador, cache, compilador, linguagem, SO, I/O e concorrência.

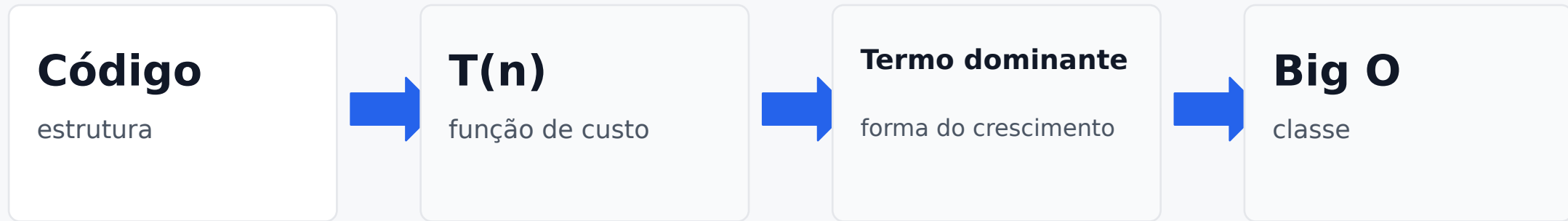
passos lógicos

Pertencem ao algoritmo e permitem comparar soluções antes de executar.

Complexidade começa na contagem de ações fundamentais.

Do código ao Big O

pipeline



A ordem assintótica é o último passo. Ela não substitui a função de custo.

Ação fundamental

primeira decisão da análise

Antes de contar, defina o que será contado.

comparação

$v[i] > 0$

soma

$s += v[i]$

chamada

$\text{fib}(n-1)$

A escolha precisa ser coerente durante toda a análise.

Sequências calibram o crescimento

constantes

```
for i in 1:n  
    a += 1  
    b += 1  
    c += 1  
end
```

As três ações dentro do laço geram uma constante multiplicativa.

$$T(n) = 3n$$

No Big O, isso vira $O(n)$, mas a constante representa trabalho real.

Constante aditiva e constante multiplicativa

$$T(n)=an+b$$

```
x = 0
y = 0
z = 0

for i in 1:n
    soma += v[i]
    cont += 1
end

media = soma / cont
```

Antes do laço: custo fixo

Dentro do laço: custo por elemento

Depois do laço: custo fixo

Exemplo aproximado: $T(n)=2n+4$

Condicionais ajustam caminhos

seleção

```
if x > 0
  som a += 1
else
  som a -= 1
end
```

**Mesmo com dois caminhos,
apenas um é executado.**

O custo costuma permanecer constante:
 $O(1)$.

Mas condicionais importam para
melhor caso, pior caso e caso médio.

O IF dentro do laço

onde o custo se repete

```
for i in 1:n
    if v[i] > 0
        cont += 1
    end
end
```

O IF é constante, mas está sendo executado n vezes.

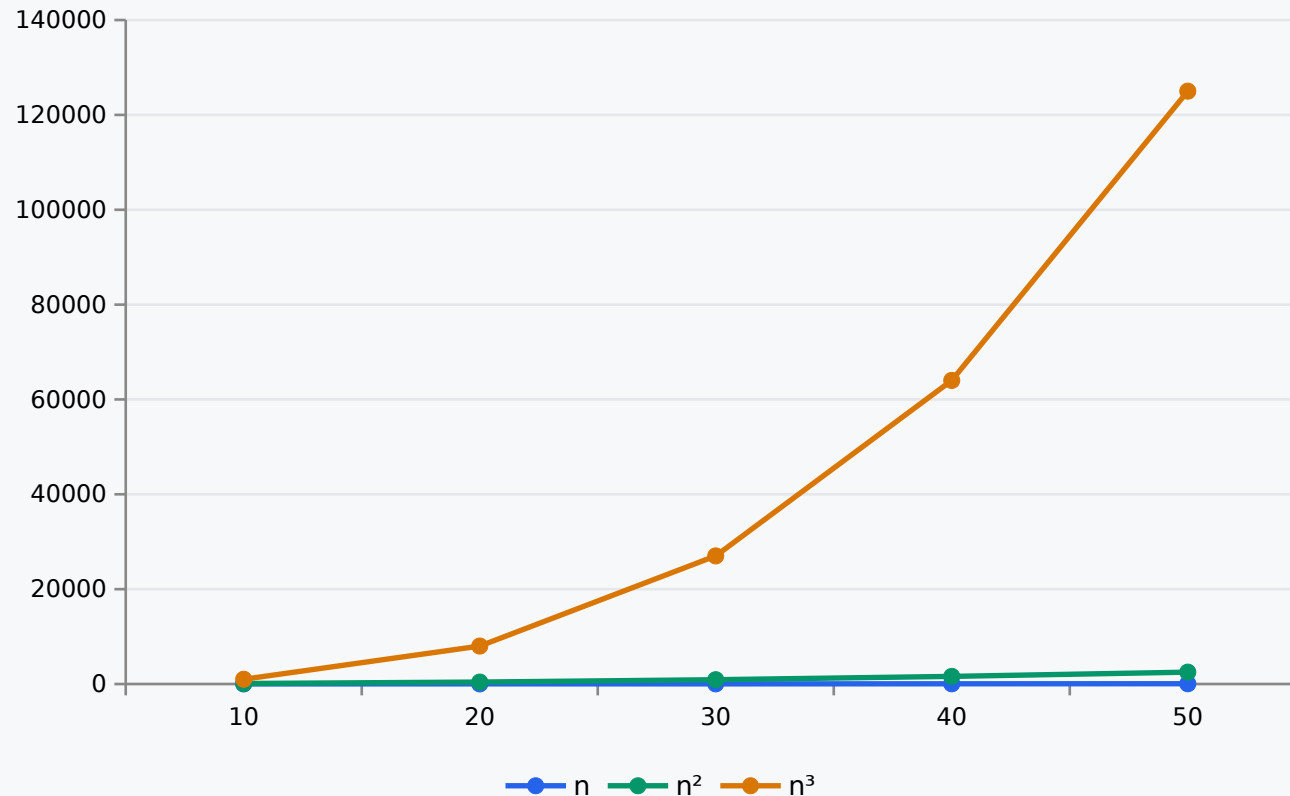
$$T(n) \approx c \cdot n$$

A pergunta prática é: quantas vezes esta decisão será tomada?

Laços multiplicam trabalho

repetição

Número de ações aproximado



Cada laço aninhado introduz uma nova dimensão de crescimento.

1 laço → $O(n)$

2 laços → $O(n^2)$

3 laços → $O(n^3)$

Laço simples: crescimento linear

exemplo

```
function soma(v)
  s = 0
  for x in v
    s += x
  end
  return s
end
```

Se há uma ação principal por elemento:

$$T(n) = n$$

Custo marginal:

$$T(n) - T(n-1) = 1$$

Laço duplo: crescimento quadrático

produto de dimensões

```
for i in 1:n
  for j in 1:n
    cont += 1
  end
end
```

O laço interno executa n vezes para cada valor de i .

$$n \cdot n = n^2$$

Custo marginal aproximado cresce com n .

Multiplicação clássica de matrizes

três índices

```
for i in 1:n
  for j in 1:n
    C[i,j] = 0
    for k in 1:n
      C[i,j] += A[i,k] * B[k,j]
    end
  end
end
```

Há n^2 posições em C.

Para cada posição, somamos n produtos.

$$T(n) \approx n^2 \cdot n = n^3$$

Este exemplo mostra como a estrutura do problema impõe crescimento.

Big O e o papel de n_0

abstração

$f(n) = O(g(n))$ quando existem $c > 0$ e n_0 tais que:

**$f(n) \leq c \cdot g(n)$
para todo $n \geq n_0$**

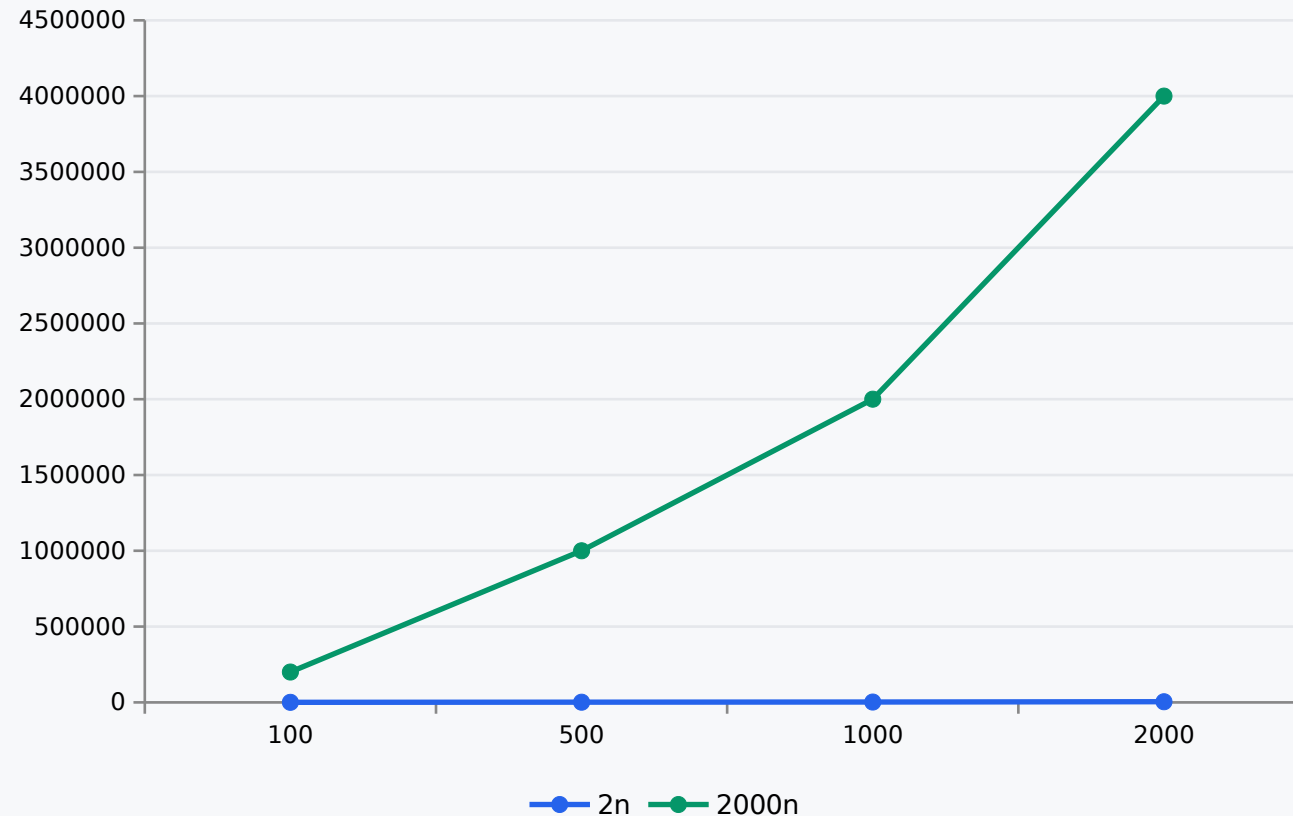
O n_0 admite que o início da curva pode ser irregular.

Big O não nega as constantes. Ele escolhe olhar para a região assintótica.

Constantes escondidas ainda ~~importam~~

eficiência x escalabilidade

Mesma classe, inclinações diferentes



Ambas são $O(n)$.

Mas a constante multiplicativa muda o custo concreto.

**Big O avalia escalabilidade;
 $T(n)$ preserva eficiência
aproximada.**

Coeficiente angular e custo marginal

ponte com cálculo

Para $T(n)=an+b$, o coeficiente a é a inclinação da reta de custo.

Derivada discreta:

$$\Delta T(n) = T(n) - T(n-1)$$

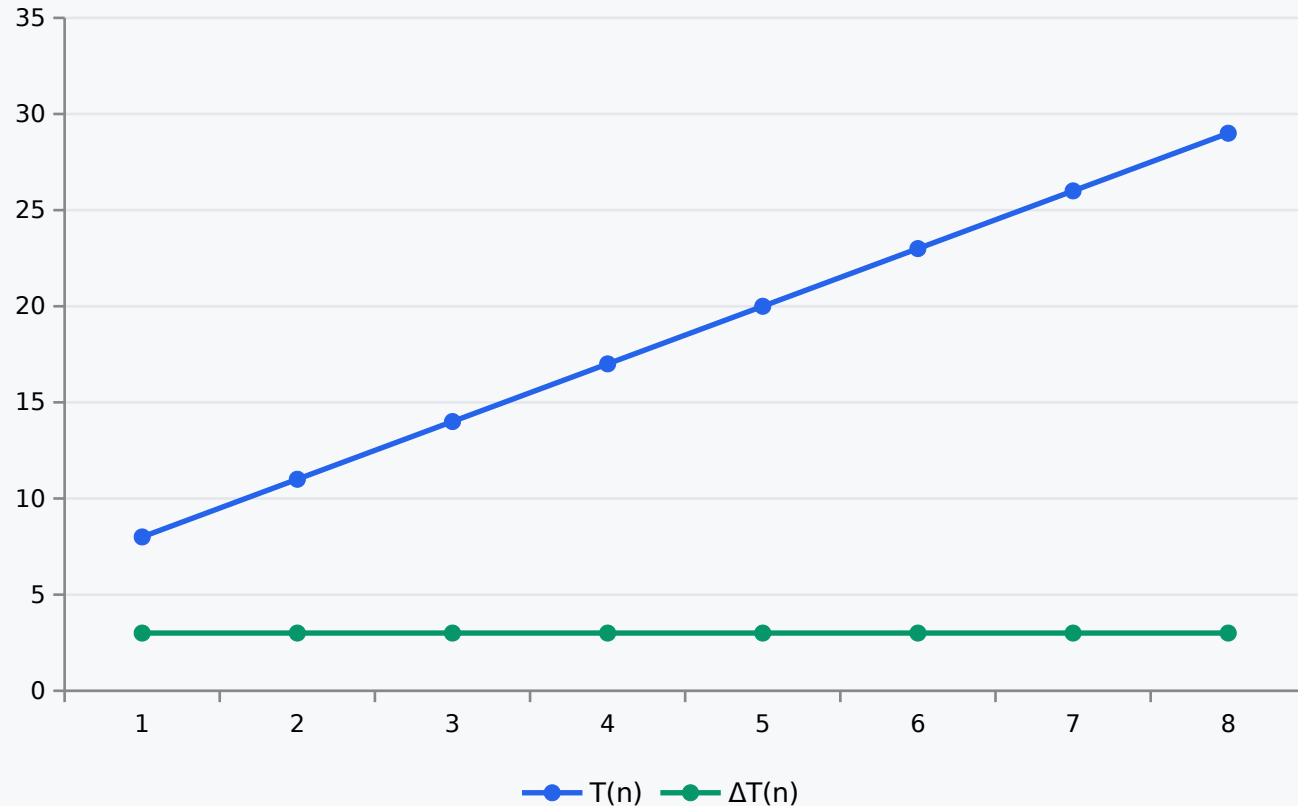
Interpretação: quanto custa adicionar mais um elemento à entrada.

Essa ideia aproxima análise de algoritmos e cálculo discreto.

Custo local estável

linear

$$T(n)=3n+5 \text{ e } \Delta T(n)=3$$



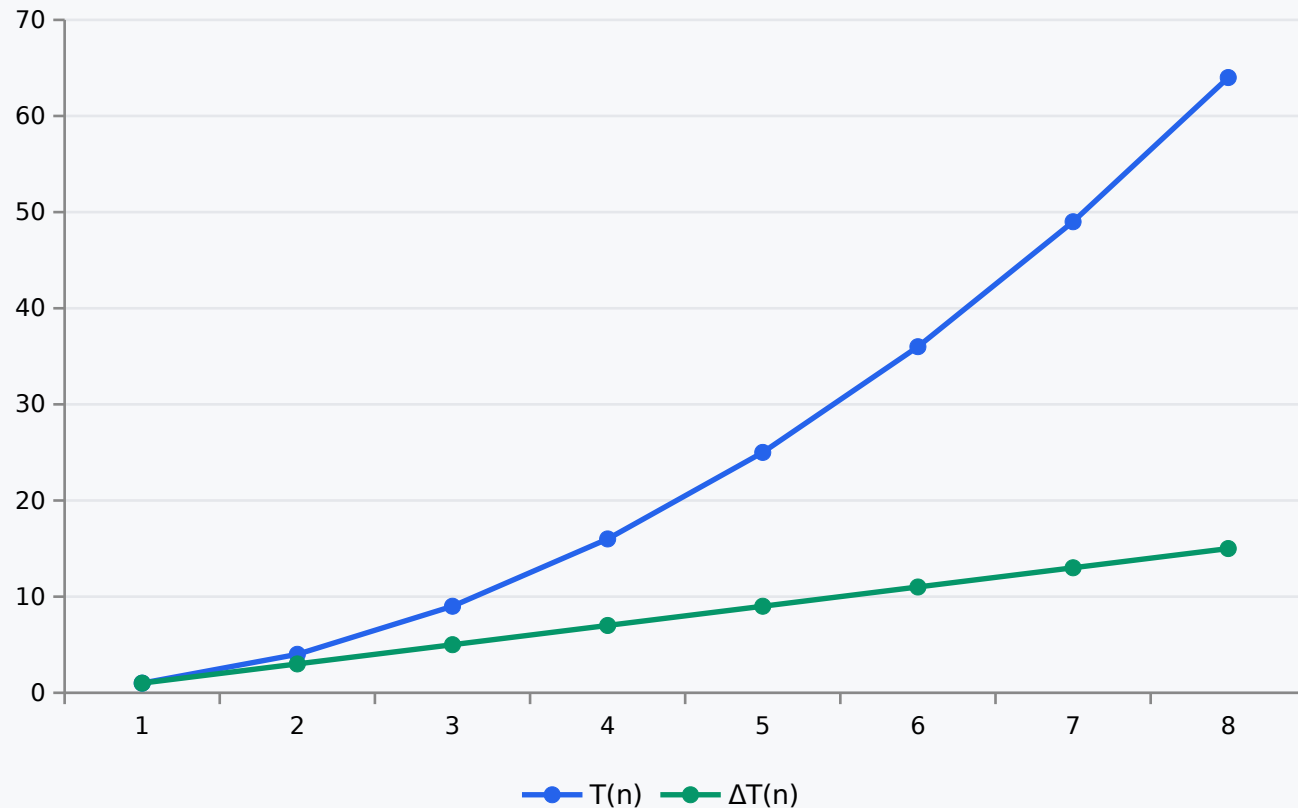
Cada novo elemento acrescenta praticamente o mesmo bloco de trabalho.

Exemplo: percorrer um vetor uma única vez.

Custo local crescente

quadrático

$$T(n)=n^2 \text{ e } \Delta T(n)=2n-1$$



O trabalho adicional por novo elemento cresce com n .

Por isso algoritmos quadráticos ficam rapidamente caros.

Fibonacci como função

recorrência

$$F(0)=0$$

$$F(1)=1$$

$$F(n)=F(n-1)+F(n-2)$$

A definição é curta, mas a implementação recursiva ingênua gera uma árvore de chamadas.

O problema não é “usar recursão”. O problema é recalcular os mesmos subproblemas.

Fibonacci recursivo ingênuo

código

```
function fib(n)
  if n <= 1
    return n
  end
  return fib(n-1) + fib(n-2)
end
```

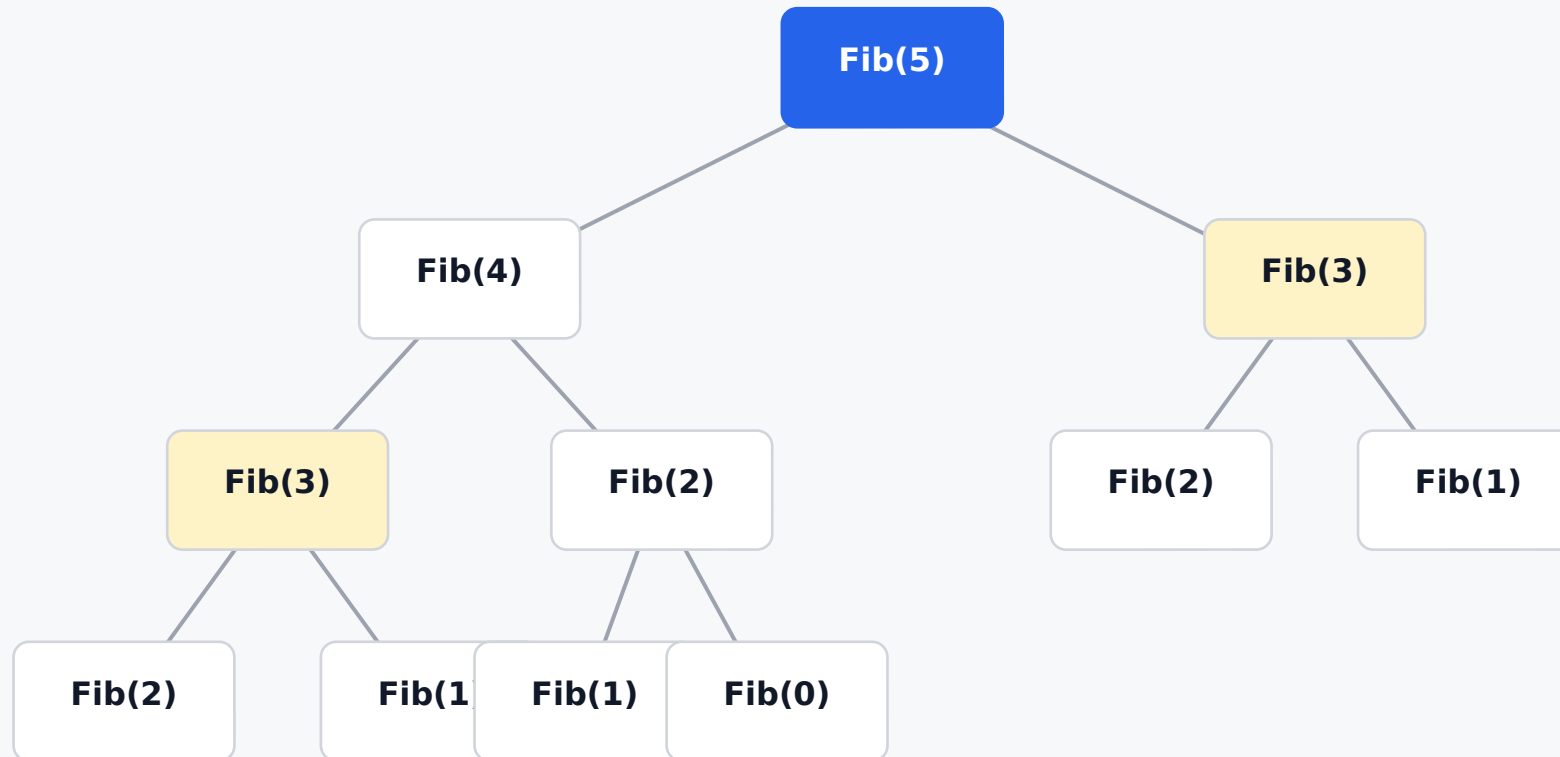
Há poucas linhas de código, mas muitas chamadas repetidas.

Linhas de código não medem complexidade.

A estrutura de chamadas é o objeto da análise.

Árvore de chamadas

recomputação visual



Fib(3) e Fib(2) aparecem mais de uma vez. Isso é recomputação.

Custo do Fibonacci ingênuo

recorrência de custo

$$T(n) = T(n-1) + T(n-2) + 1$$

A soma das duas chamadas gera uma expansão em árvore.

Crescimento aproximado:

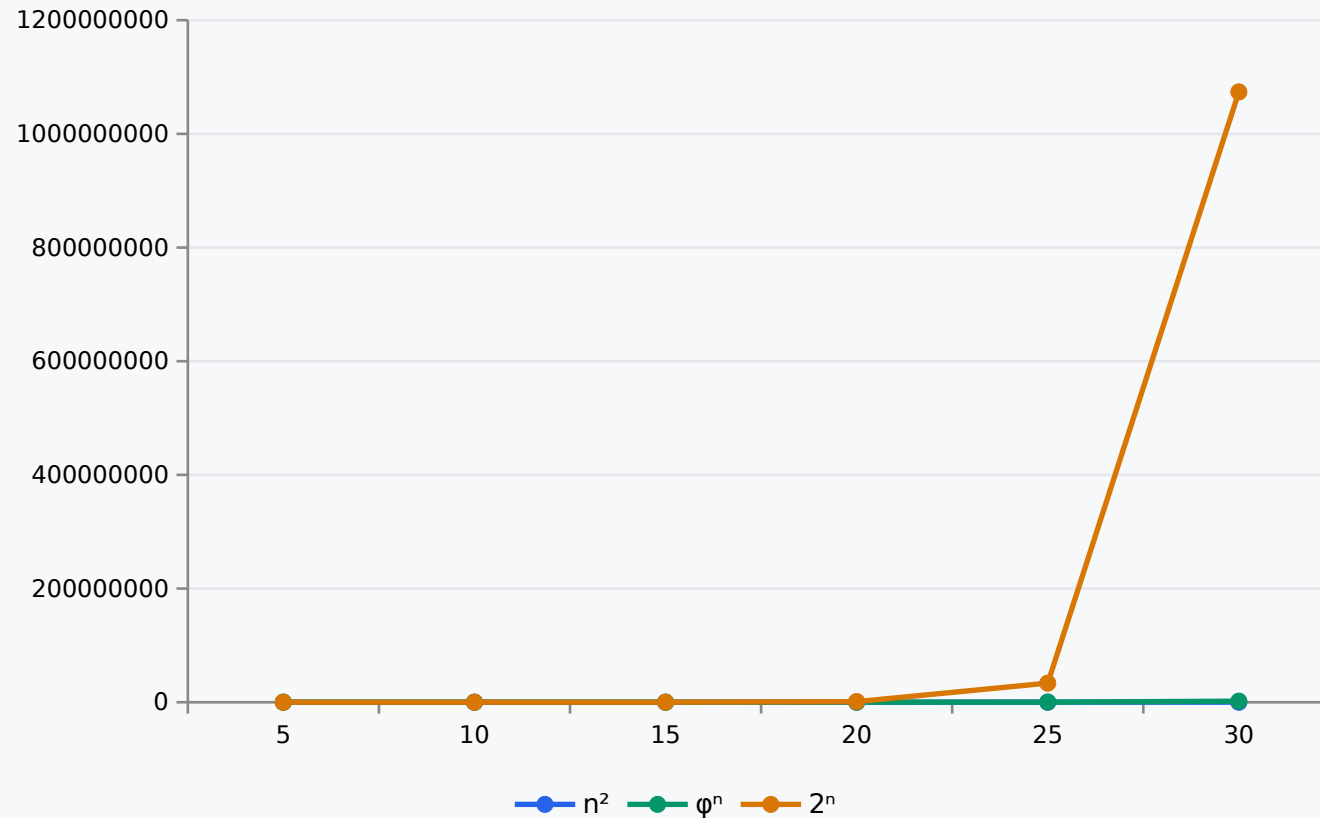
$$O(\varphi^n)$$

$$\varphi = (1 + \sqrt{5})/2 \approx 1,618$$

Polinomial versus exponencial

mudança estrutural

Crescimento comparado



Em n^2 , a variável está na base.

Em φ^n , a variável está no expoente.

Essa diferença explica a explosão.

Recursão não é sinônimo de explosão

precisão conceitual

linear

$$T(n) = T(n-1) + 1$$
$$O(n)$$

logarítmica

$$T(n) = T(n/2) + 1$$
$$O(\log n)$$

com sobreposição

$$T(n) = T(n-1) + T(n-2) + 1$$
$$O(\varphi^n)$$

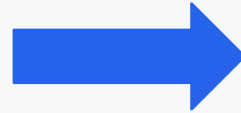
A estrutura dos subproblemas determina o crescimento.

Programação dinâmica

eliminar recomputação

Sem memória

O mesmo subproblema é resolvido várias vezes.



Com memoização

Cada estado $F(i)$ é calculado uma única vez e armazenado.

Custo: $O(n)$

Fibonacci iterativo

mesma função, outra estratégia

```
function fib_iter(n)
  a, b = 0, 1
  for i in 2:n
    a, b = b, a + b
  end
  return n == 0 ? 0 : b
end
```

O algoritmo percorre os estados de 0 até n.

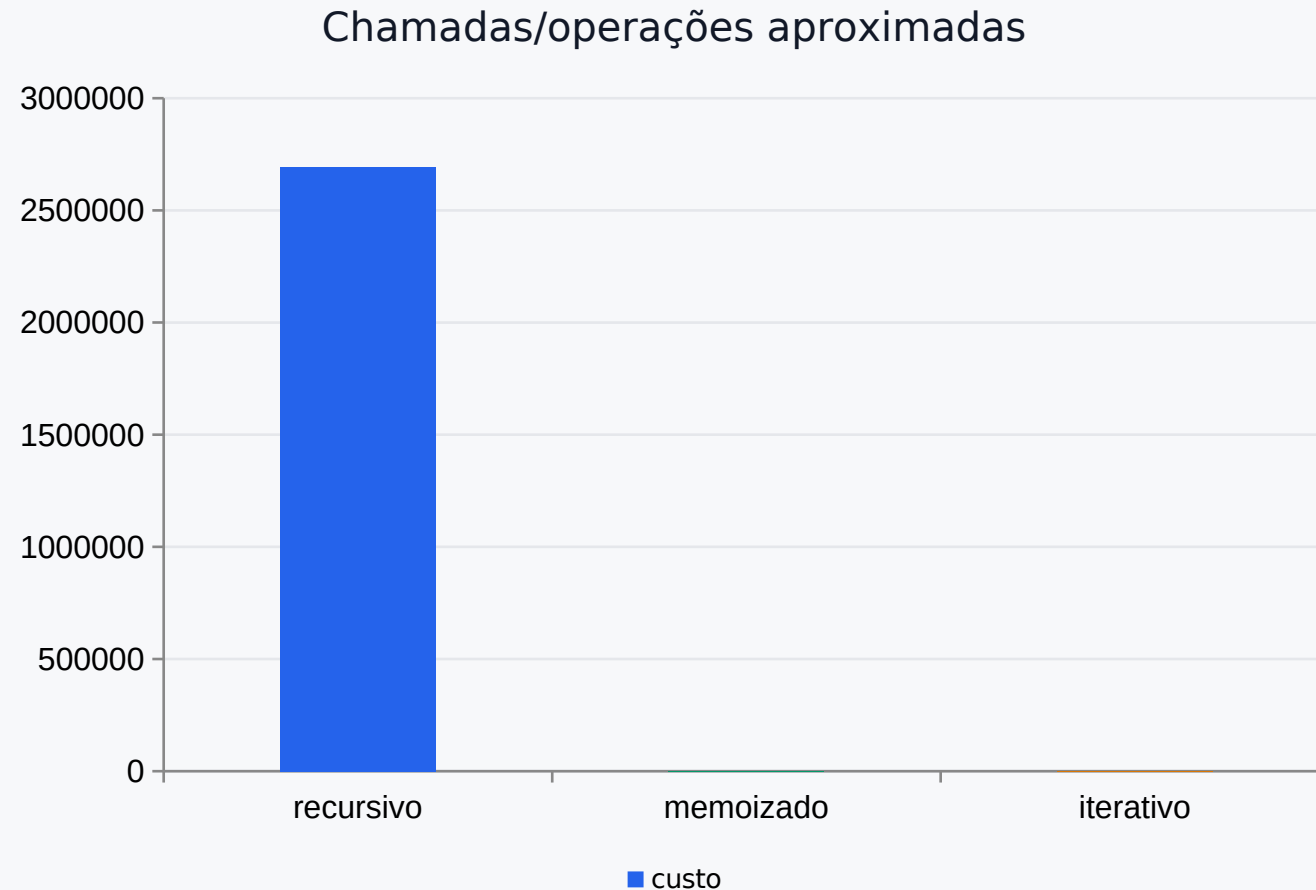
Tempo: $O(n)$

Memória: $O(1)$

Não há recomputação.

Comparando estratégias para Fibonacci

n = 30



A programação dinâmica não muda a definição de Fibonacci.

Ela muda a organização do cálculo.

Resultado: de exponencial para linear.

Julia + Pluto

por que usar na aula

Pluto permite transformar complexidade em experimento visual.

Sliders

alteram n em
tempo real

Gráficos

mostram
crescimento

Código

gera a própria
evidência

Estrutura do notebook Pluto

roteiro executável

O notebook acompanha a apresentação e contém quatro experimentos.

1. Contagem de operações em laços
2. Curvas n , n^2 e n^3
3. Fibonacci: recursivo, memoizado e iterativo
4. Custo marginal $\Delta T(n)$

```
using PlutoUI, Plots
@bind n Slider(1:100, default=20)
plot(ns, [ns, ns.^2, ns.^3])
```

Experimento 1: laços

contar, não cronometrar

```
ops_linear(n) = n
ops_quadrático(n) = n^2
ops_cúbico(n) = n^3

[n ops_linear(n) ops_quadrático(n) ops_cúbico(n)]
```

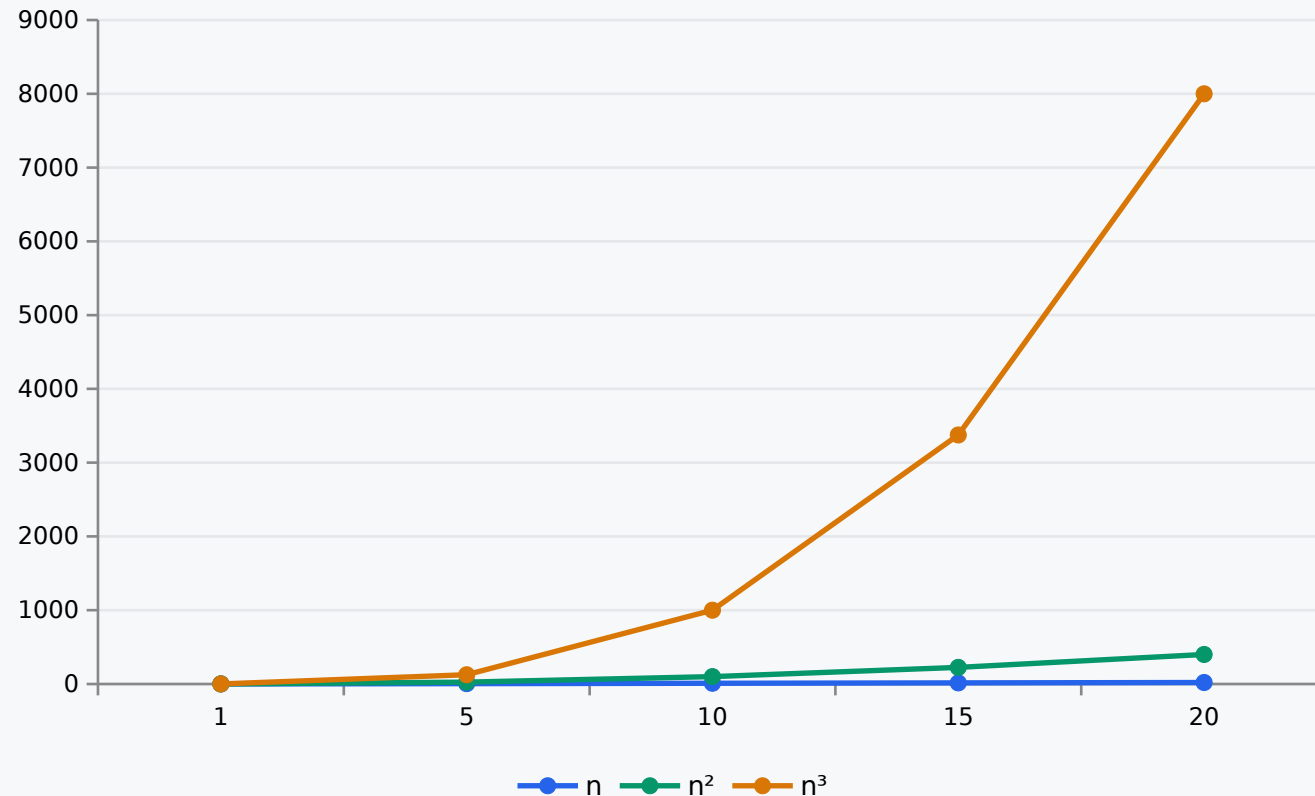
O aluno altera n e observa a diferença entre crescimento linear, quadrático e cúbico.

A tabela é uma ponte entre código e função de custo.

Experimento 2: curvas

visualização

Crescimento gerado no Pluto



A visualização torna perceptível a diferença entre “funcionar” e “escalar”.

O gráfico é gerado a partir das funções de custo.

Experimento 3: Fibonacci

contagem de chamadas

```
calls_naive(n) = n <= 1 ? 1 : calls_naive(n-1) + calls_naive(n-2) + 1  
fib_iter_ops(n) = max(n, 1)
```

Para evitar travar a aula, o notebook modela a quantidade de chamadas pela recorrência de custo.

O objetivo não é esperar o algoritmo terminar, mas entender por que ele explode.

Experimento 4: custo marginal

$\Delta T(n)$

```
delta(T, n) = T(n) - T(n-1)
```

```
linear(n) = 3n + 5
```

```
quadratica(n) = n^2
```

```
plot(ns[2:end], [delta.(Ref(linear), ns[2:end]),  
                  delta.(Ref(quadratica), ns[2:end])])
```

A derivada discreta mostra o custo de aumentar a entrada em uma unidade.

Essa é a ponte com coeficiente angular, inclinação e custo marginal.

Atividade em sala

estimativa durante a escrita

Antes de executar, peça que os alunos respondam:

1. Qual é a ação fundamental?
2. Quantas vezes ela ocorre?
3. A sequência gera constante aditiva ou multiplicativa?
4. O IF muda a ordem ou apenas o caminho?
5. Se dobrar n , o trabalho dobra, quadruplica ou cresce mais?

Big O não é o ponto de partida

01

Big O não é o ponto de partida

O ponto de partida é a função de custo $T(n)$ do algoritmo — o Big O resume apenas o seu termo dominante.

02

Laços moldam o crescimento

Sequências calibram constantes; laços aninhados multiplicam o crescimento — é aí que nascem $O(n^2)$ e $O(n \log n)$.

03

Programação dinâmica elimina recomputação

Guardar resultados de subproblemas troca uma explosão exponencial, como no Fibonacci ingênuo, por custo linear.



A SEGUIR — AULA 15 de 15

Encerramento e Discussão

Referências

RAABE, André; ZORZO, Avelino F.; BLIKSTEIN, Paulo. Computação na Educação Básica: Fundamentos e Experiências. Porto Alegre: Penso, 2020. Ebook. ISBN 9786581334048.

RIBEIRO, João Araújo. Introdução à Programação e aos algoritmos. Rio de Janeiro: LTC, 2019. ISBN 978-85-216-3640-3.